Rolling joinをSASで考えてみる

イーピーエス株式会社　森岡裕

『**名前をつけてやる**』は、 日本のロックバンド・スピッツの2作目のオリジナルアルバム。1991年11月25日

名盤！！

「名前をつける」＝存在を肯定する行為

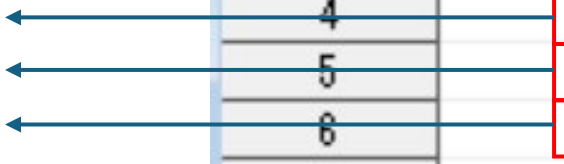
正体不明で不器用な感情や存在を、排除せず、笑わず、「それでもいい」と認めてあげること

VIEWTABLE: Work.A

	ID	TIME
1	1	-1
2	1	2
3	1	5
4	1	10
5	2	2
6	2	5
7	2	10
8	2	16

VIEWTABLE: Work.B

	ID	TIME	VAL
1	1	4	B
2	1	1	A
3	1	6	C
4	1	10	D
5	2	2	E
6	2	5	F
7	2	10	G
8	2	10	H



例えばAにBから，IDとTIMEをkeyとして，B.VALの値を取得したい場合を考える
ID=1のTIME=10　やID=2のTIME=2,5などはAにもBにも存在する値なのでSQLでいうLEFT JOINで簡単にVALを取得できるだろう

VIEWTABLE: Work.A

	ID	TIME
1	1	-1
2	1	2
3	1	5
4	1	10
5	2	2
6	2	5
7	2	10
8	2	16

VIEWTABLE: Work.B

	ID	TIME	VAL
1	1	4	B
2	1	1	A
3	1	6	C
4	1	10	D
5	2	2	E
6	2	5	F
7	2	10	G
8	2	10	H

ID=1のTIME=2について，データセットBには「**マッチするIDの中でTIMEでマッチする値が存在しない**」
同値のTIMEが存在しない場合に，**TIMEにおいて，過去方向で最も近い値をもつレコードをマッチする**
上記の場合であれば，2より過去で，最も大きい値は1なので，1でマッチさせる

このように特定の結合キーのある変数が完全一致しない場合に，条件に当てはまるレコードを代わりにマッチさせる結合について考える

VIEWTABLE: Work.A

	ID	TIME
1	1	-1
2	1	2
3	1	5
4	1	10
5	2	2
6	2	5
7	2	10
8	2	16

VIEWTABLE: Work.B

	ID	TIME	VAL
1	1	4	B
2	1	1	A
3	1	6	C
4	1	10	D
5	2	2	E
6	2	5	F
7	2	10	G
8	2	10	H

VIEWTABLE: Work.C1

	ID	TIME	VAL
1	1	-1	
2	1	2	A
3	1	5	B
4	1	10	D
5	2	2	E
6	2	5	F
7	2	10	H
8	2	16	H

IDが完全一致する中で，A.TIME<=B.TIMEとなる最大のTIMEを持つレコードをマッチする
Bに同TIMEが重複して存在する場合は元のオブザベーションが先の方をとる

という処理を実装すると，結果は右のようになる

だけど
その処理には名前がなかった

その処理には名前がなかった

はっきり言って　ありふれた条件付き結合処理です.

時系列データ処理の実務的要請から段階的に形成され、後に名前が与えられた概念

ありふれた処理概念が名前を獲得することでどのようになるのかを歴史的流れに沿って整理します

1. 時系列データ処理の黎明期（1960–80年代）

本質的な課題

•「ある時点の観測値に対し、直前（または最近）の情報を結び付けたい」
•金融、製造、臨床、センサーデータで普遍的に発生

当時の実装

•明示的な概念名は存在しない
•以下のような処理として実現されていた：

- Fortran / COBOL での逐次走査
- 「最後に見た値を保持する」ロジック
- 手続き型の *stateful processing*

ポイント！
この段階では「join」ではなく「時系列補完・参照」として扱われていました。

2. RDB・SQL時代：非等価結合としての発展（1980–90年代）

SQLにおける前身概念
•Non-equi join（非等価結合）

```
SELECT *
FROM A
JOIN B
  ON B.date <= A.date
```

問題点
•結果が多対多になりやすい
•「直前1件」を選ぶには：
  - サブクエリ
  - GROUP BY + MAX
  - 相関サブクエリ

👉 論理的には可能だが、非効率・冗長

この時点で：
•「時点Aに対し、Bの最新レコードを結合する」というパターンは確立
•しかし名前も抽象化もまだ不十分

3. SAS・統計処理系での定式化（1990–2000年代）

SAS DATA step における考え方
•BY-group processing
•retain / lag / last.変数
•hash object（2000年代）

例（概念的）：
•Visit日に対し、直前の投与量を保持
•SDTM/ADaM で頻出

ここでも：
•「XXXXXX」という特定の名称は使われない
•しかし実務的には完全に同一の処理

特に臨床領域では：
•「**as of date**」
•「**most recent prior**」
という言い回しが主流

4. data.table による命名と概念の明確化（2010年前後）

転換点：R / data.table
Matt Dowle による data.table 開発（2006–）
Rolling Join の明示的導入

- `roll=TRUE`
- `roll=Inf`
- `roll=-Inf`

例：

```
DT2[DT1, on="date", roll=TRUE]
```

ここで起きたこと

- それまで暗黙的だった処理が：
  - **joinの一種として定義**
  - **rolling（転がす）という直感的比喩で命名**
- 「等価結合＋補正」ではなく

👉 **独立したjoin操作として概念化**

この時点で初めて：

- **rolling join ＝ 時系列最近傍結合**

という認識が広まる

Rolling Join - data.table

```
#Rolling Join
dt_query <- data.table(time = c(2,6,9))
```

	time
1	2
2	6
3	9

```
dt_time <- data.table(time = c(1,5,10)
, value = c("a","b","c"))
```

	time	value
1	1	a
2	5	b
3	10	c

```
#Join the most recent past value
dt_out5 <- dt_time[dt_query, on=.(time), roll=TRUE]
```

	time
1	2
2	6
3	9

	time	value
1	1	a
2	5	b
3	10	c



	time	value
1	2	a
2	6	b
3	9	b

Rolling Join - data.table

Rolling join is an ordered join that fills in matches using the closest previous (or next) key value, when an exact match is not found.

For each value in the query table, find the closest matching key in the source table that does not exceed it (when roll=TRUE).

In data.table's Rolling Join, when there are duplicate key values,the row that is matched depends on the mult argument:by default, the first matching row is used (mult = "first"),while setting mult = "last" makes it use the last one.

Option	Description
roll = TRUE	**Carry the most recent past value forward (LOCF)**
roll = Inf	**Carry the most recent past value forward (LOCF)**
roll = -Inf	**Carry the next future value backward (NOCB)**
roll = "nearest"	**Use whichever value is closest (past or future)**

5. その後の一般化・他言語への波及（2015年以降））

SQL

- Window関数：
  - `last_value()`
  - `row_number() over (partition by … order by …)`
- **LATERAL JOIN**
- **ASOF JOIN**（ClickHouse, DuckDB）

Python / pandas

- `merge_asof()`
  - 明示的に **as-of join** と命名

概念史のポイント

1.処理は昔から存在
2.名前と抽象化が後から追いついた
3.data.table が初めて，名前を与え，joinとして定義し高速実装を提供した

一言で言うと
Rolling join は
時系列処理の職人芸を、宣言的データ操作に昇華した概念

そんなRolling Joinに森岡なりに敬意を表して

Rolling Joinを，森岡なりに**Rolling Match**として再定義

Rolling Matchは以下の機能を持つ

①**BACK**⇒完全一致しない場合は**過去方向**に走査して最も近い値でマッチする
②**FORWARD**⇒完全一致しない場合は**未来方向**に走査して最も近い値でマッチする
③**NEAREST**⇒完全一致しない場合は**過去未来方向**に走査して最も近い値でマッチする

Rolling Matchは以下の特性を持つ

・JoinではなくMatchであるため，1対多をによるレコード拡大を認めない
・走査の許容幅を設定可能，たとえばBACKなら，過去3日以内の差をマッチ対象にするなど，NEARESTなら　-3日から＋5日までといった指定も可能
・ NEARESTについて，たとえば，-3日と＋３日は起点から等距離であるがその際に前を採用するか，後を採用するかは，任意に指定可能とする

```
data C;
set A;
if 0 then set B;
if _N_ = 1 then do;
length ___row_number ___TIME 8.;
call missing(of ___row_number ___TIME min_diff hash_diff target_row_number);
rc=dosubl("data htemp0/view=htemp0;set B curobs=_cur; cobs=_cur;run;
proc sql noprint;
create view htemp as select *, cobs as ___row_number,time as ___time from htemp0
order by ID, TIME;
quit;"
);
declare hash h1(dataset:"htemp",multidata:"Y");
h1.definekey("ID");
h1.definedata("___TIME","___row_number");
h1.definedone();
declare hash h2(dataset:"htemp");
h2.definekey("___row_number");
h2.definedata("VAL");
h2.definedone();
end;
call missing(of ___row_number ___TIME);
call missing(min_diff, target_row_number);
do while(h1.do_over()=0);
 if n(of TIME ___TIME) =2 then hash_diff = TIME - ___TIME;

 if ^missing(hash_diff) and 0<=hash_diff then do;
    if missing(min_diff) or hash_diff <= min_diff then do;
          if hash_diff =min_diff and  ___row_number < target_row_number  then do;
            target_row_number =___row_number;
          end;
          else do;
            target_row_number =___row_number;
          end;
          min_diff = hash_diff;
    end;
 end;

end;

___row_number=target_row_number;
if h2.find() ne 0 then do;
 call missing(of VAL);
end;

drop ___row_number target_row_number rc min_diff hash_diff ___TIME;
run;
```

たとえばBACKを，平コードで実装すると右のようになる

肝なのは，マッチするレコードの特定と

そこから変数を取得する部分のハッシュオブジェクトを2つにわけていること

VIEWTABLE: Work.A

	ID	TIME
1	1	-1
2	1	2
3	1	5
4	1	10
5	2	2
6	2	5
7	2	10
8	2	16

VIEWTABLE: Work.B

	ID	TIME	VAL
1	1	4	B
2	1	1	A
3	1	6	C
4	1	10	D
5	2	2	E
6	2	5	F
7	2	10	G
8	2	10	H

一度，取得元に対して
オブザベーション番号を付与した
Viewをdosubl関数で作り
それをハッシュにいれて
マルチキーで定義して
Do_overメソッドでもっとも近い
地点のオブザベーション番号を
特定する.
そしてそれをキーにして
別途値を取得する



VIEWTABLE: Work.Htemp

	ID	TIME	VAL	cobs	___row_number	___time
1	1	1	A	2	2	1
2	1	4	B	1	1	4
3	1	6	C	3	3	6
4	1	10	D	4	4	10
5	2	2	E	5	5	2
6	2	5	F	6	6	5
7	2	10	G	7	7	10
8	2	10	H	8	8	10

万人が使える形に実装してこそ意味が生まれるので

マクロ化しました

```
%rolling_match(
Master=,
Key=,
Rollvar=,
Rolltype=BACK,
Roll_back_limit=,
Roll_forward_limit=,
Var=,
Wh=,
Nearest_tie_dir=BACK,
dupWARN=Y);
```

•master=（必須）
検索対象となるマスターデータセット（例：B）。
このデータセットからマッチした値が取得されます。

•key=（必須）
マッチンググループを定義する 1 つ以上のキー変数（スペース区切り）。
例：key=ID ／ key=STUDYID USUBJID ／ key=SITEID SUBJID VISITNUM

•rollvar=（必須）
ローリングロジックに使用される時間型変数。通常は数値型（例：datetime、date、visit day、time）。
現在の DATA ステップのデータセットおよびマスターデータセットの両方に存在している必要があります。

•rolltype=（任意、デフォルト=BACK）
ローリングの方向／戦略。指定可能な値：BACK、FORWARD、NEAREST。
-- BACK
現在値以前で最も近い値を選択します（master.rollvar <= current.rollvar）。
距離は diff = current.rollvar - master.rollvar（diff >= 0）として定義されます。
-- FORWARD
現在値以後で最も近い値を選択します（master.rollvar >= current.rollvar）。
距離は diff = current.rollvar - master.rollvar（diff <= 0）として定義されます。
--NEAREST
前後いずれの方向でも最も距離の近い値を選択します（abs(diff) を最小化）。
過去と未来の候補が同距離の場合は、nearest_tie_dir= で優先方向を制御できます。

•roll_back_limit=（任意、デフォルト空＝制限なし）
BACK および NEAREST における最大の過去方向距離。
diff = current - master（diff >= 0）として解釈され、
0 <= diff <= roll_back_limit を満たす候補のみが対象となります。

•roll_forward_limit=（任意、デフォルト空＝制限なし）
FORWARD および NEAREST における最大の未来方向距離。
diff = current - master（diff <= 0）として解釈され、
-roll_forward_limit <= diff <= 0 を満たす候補のみが対象となります。

•var=（必須）
マッチしたマスターレコードから取得する 1 つ以上の変数（スペース区切り）。
これらの変数は現在の DATA ステップの PDV 上で作成または上書きされます。
例：var=VAL ／ var=VAL FLAG ／ var=LABVAL LABUNIT

•wh=（任意、デフォルト空）
内部参照ビュー作成時にマスターデータセットへ適用される WHERE 条件。
%nrbquote() などのマクロマスキング関数と併用することを前提としています。
条件は DATA ステップの WHERE 文にそのまま挿入されます。

実行例

```
data A;
ID=1;TIME=-1;output;
ID=1;TIME=2;output;
ID=1;TIME=5;output;
ID=1;TIME=10;output;
ID=2;TIME=2;output;
ID=2;TIME=5;output;
ID=2;TIME=10;output;
ID=2;TIME=16;output;
run;
```

VIEWTABLE: Work.A

	ID	TIME
1	1	-1
2	1	2
3	1	5
4	1	10
5	2	2
6	2	5
7	2	10
8	2	16

```
data B;
ID=1;TIME=4;VAL="B";output;
ID=1;TIME=1;VAL="A";output;
ID=1;TIME=6;VAL="C";output;
ID=1;TIME=10;VAL="D";output;
ID=2;TIME=2;VAL="E";output;
ID=2;TIME=5;VAL="F";output;
ID=2;TIME=10;VAL="G";output;
ID=2;TIME=10;VAL="H";output;
run;
```

VIEWTABLE: Work.B

	ID	TIME	VAL
1	1	4	B
2	1	1	A
3	1	6	C
4	1	10	D
5	2	2	E
6	2	5	F
7	2	10	G
8	2	10	H

VIEWTABLE: Work.A

	ID	TIME
1	1	-1
2	1	2
3	1	5
4	1	10
5	2	2
6	2	5
7	2	10
8	2	16

VIEWTABLE: Work.B

	ID	TIME	VAL
1	1	4	B
2	1	1	A
3	1	6	C
4	1	10	D
5	2	2	E
6	2	5	F
7	2	10	G
8	2	10	H

```
data C1;
set A;
%rolling_match(
master=B,key=ID,rollvar=TIME,
rolltype=BACK,var=VAL
);
run;
```

VIEWTABLE: Work.C1

	ID	TIME	VAL
1	1	-1	
2	1	2	A
3	1	5	B
4	1	10	D
5	2	2	E
6	2	5	F
7	2	10	H
8	2	16	H

VIEWTABLE: Work.A

	ID	TIME
1	1	-1
2	1	2
3	1	5
4	1	10
5	2	2
6	2	5
7	2	10
8	2	16

VIEWTABLE: Work.B

	ID	TIME	VAL
1	1	4	B
2	1	1	A
3	1	6	C
4	1	10	D
5	2	2	E
6	2	5	F
7	2	10	G
8	2	10	H

```
data D1;
set A;
%rolling_match(
master=B,key=ID,rollvar=TIME,
rolltype=FORWARD,var=VAL
);
run;
```

VIEWTABLE: Work.D1

	ID	TIME	VAL
1	1	-1	A
2	1	2	B
3	1	5	C
4	1	10	D
5	2	2	E
6	2	5	F
7	2	10	H
8	2	16	

VIEWTABLE: Work.A

	ID	TIME
1	1	-1
2	1	2
3	1	5
4	1	10
5	2	2
6	2	5
7	2	10
8	2	16

VIEWTABLE: Work.B

	ID	TIME	VAL
1	1	4	B
2	1	1	A
3	1	6	C
4	1	10	D
5	2	2	E
6	2	5	F
7	2	10	G
8	2	10	H

```
data E1;
set A;
%rolling_match(
master=B,key=ID,rollvar=TIME,
rolltype=NEAREST,var=VAL
);
run;
```

VIEWTABLE: Work.E1

	ID	TIME	VAL
1	1	-1	A
2	1	2	A
3	1	5	B
4	1	10	D
5	2	2	E
6	2	5	F
7	2	10	G
8	2	16	G

https://github.com/PharmaForest/mergex

mergex

MERGEX is a package that enables non-standard or unconventional joins not easily handled or supported by standard SAS syntax. It currently implements variable-name conflict-safe joins, and rolling joins .



PharmaForestのmergexパッケージ内に
Rolling_matchマクロとして公開していて
使用例もかなり豊富に書いたので見てみてください

1) Basic BACK (default) with no distance limit

```
data out_back;
  set A;
  %rolling_match(master=B, key=ID, rollvar=TIME, rolltype=BACK, var=VAL VAL2);
run;
```

VIEWTABLE: Work.Out_back

	ID	TIME	VAL	VAL2
1	1	-1		.
2	1	2	A	2
3	1	5	B	1
4	1	10	D	4
5	2	2	E	10
6	2	5	F	10
7	2	10	H	40
8	2	16	H	40

2) BACK with a backward distance limit (e.g., within 3 units)

```
data out_back_lim;
  set A;
  %rolling_match(master=B, key=ID, rollvar=TIME, rolltype=BACK, roll_back_limit=3, var=VAL);
run;
```

VIEWTABLE: Work.Out_back_lim

	ID	TIME	VAL
1	1	-1	
2	1	2	A
3	1	5	B
4	1	10	D
5	2	2	E
6	2	5	F
7	2	10	H
8	2	16	

3) FORWARD with a forward distance limit (e.g., within 5 units)

```
data out_fwd_lim;
  set A;
  %rolling_match(master=B, key=ID, rollvar=TIME, rolltype=FORWARD, roll_forward_limit=5, var=VAL);
run;
```

VIEWTABLE: Work.Out_fwd_lim

	ID	TIME	VAL
1	1	-1	A
2	1	2	B
3	1	5	C
4	1	10	D
5	2	2	E
6	2	5	F
7	2	10	H
8	2	16	

まとめ

統計解析理論ができるのがエライと思って，データハンドリングを軽視してる人は総じてクズである

データハンドリングの理論も立派な学問分野であり，結合について考えることも学問である

日常的に行っている名もなき処理について，今一度改めて考えて，定義する
定義した後，再度，実装してみると，そこから新しい分野が開花したりする．

つまり，

「名前をつけてやる」という態度は
世界を肯定し，前進させる力である

おまけ

- do_overメソッドやハッシュ反復子オブジェクトの基礎

do_over　Method

ハッシュオブジェクトの重複キーのリスト内を移動します。

ハッシュオブジェクトが単一のキーに対して複数の値を持つ場合、DO_OVERメソッドをDOループの反復内で使用して、重複キー間を移動します。DO_OVERメソッドは最初のメソッド呼び出しでキーを読み込み、キーが最後に到達するまで重複キーリストを移動しつづけます。

```
data AE;
SUBJID="001";AENO="1";output;
SUBJID="001";AENO="2";output;
SUBJID="001";AENO="3";output;
SUBJID="002";AENO="1";output;
SUBJID="003";AENO="2";output;
run;
```

	SUBJID	AENO
1	001	1
2	001	2
3	001	3
4	002	1
5	003	2

```
data CM;
SUBJID="001";CMTRT="Drug A";ACNAENO="1,2";output;
SUBJID="001";CMTRT="Drug B";ACNAENO="1";output;
SUBJID="001";CMTRT="Drug C";ACNAENO="3,2";output;
SUBJID="003";CMTRT="Drug D";ACNAENO="2";output;
SUBJID="003";CMTRT="Drug E";ACNAENO="1,2";output;
run;
```

	SUBJID	CMTRT	ACNAENO
1	001	Drug A	1,2
2	001	Drug B	1
3	001	Drug C	3,2
4	003	Drug D	2
5	003	Drug E	1,2

	SUBJID	AENO	USED_DRUG
1	001	1	Drug A,Drug B
2	001	2	Drug A,Drug C
3	001	3	Drug C
4	002	1	
5	003	2	Drug D,Drug E

有害事象に使われたCMTRTを取得したいといったケース

```
data wk1;
length SUBJID AENO $200. USED_DRUG $1000.;
 set AE;
 if 0 then set CM;
 if _N_ = 1 then do;
    declare hash
    h1(dataset:"CM",multidata:"Y");
    h1.definekey("SUBJID");
    h1.definedata("CMTRT","ACNAENO");
    h1.definedone();
  end;
call missing(of ACNAENO CMTRT);
 do while(h1.do_over() = 0);
    if ^missing(ACNAENO) then do;
      do i = 1 to count(ACNAENO,",") + 1;
        if AENO = scan(ACNAENO,i,",") then
         USED_DRUG=catx(",", USED_DRUG, CMTRT);
      end;
    end;
 end;
keep SUBJID AENO USED_DRUG;
run;
```

Multidata:'Y'でハッシュオブジェクト内にkeyを重複して存在させる

わざと一意にならないkey条件する．AEとつなぐための条件がSUBJIDのみなので.

Keyがある限りfindメソッドが動くイメージ

```
 . . . .
do while(h1.do_over() = 0);
   . . . .
end;
```

ハッシュオブジェクト

	SUBJID	CMTRT	ACNAENO
1	001	Drug A	1,2
2	001	Drug B	1
3	001	Drug C	3,2
4	003	Drug D	2
5	003	Drug E	1,2

	SUBJID	AENO
1	001	1
2	001	2
3	001	3
4	002	1
5	003	2

Setしたデータセットの1obsごとにkey（SUBJID）でハッシュオブジェクト内が検査され該当するすべてのSUBJIDをもつハッシュ内のデータがfindメソッドで取得される．ヒットしなくなるまで．

```
data wk1;
length SUBJID AENO $200. USED_DRUG $1000.;
 set AE;
 if 0 then set CM;
 if _N_ = 1 then do;
    declare hash h1(dataset:"CM",multidata:"Y");
    h1.definekey("SUBJID");
    h1.definedata("CMTRT","ACNAENO");
    h1.definedone();
  end;
call missing(of ACNAENO CMTRT);
if h1.find()= 0 then do;
    if ^missing(ACNAENO) then do;
      do i = 1 to count(ACNAENO,",") + 1;
        if AENO = scan(ACNAENO,i,",") then
       USED_DRUG=catx(",", USED_DRUG, CMTRT);
      end;
    end;
do while(h1.find_next() = 0);
    if ^missing(ACNAENO) then do;
      do i = 1 to count(ACNAENO,",") + 1;
        if AENO = scan(ACNAENO,i,",") then
       USED_DRUG=catx(",", USED_DRUG, CMTRT);
      end;
    end;
 end;
end;
keep SUBJID AENO USED_DRUG;
run;
```

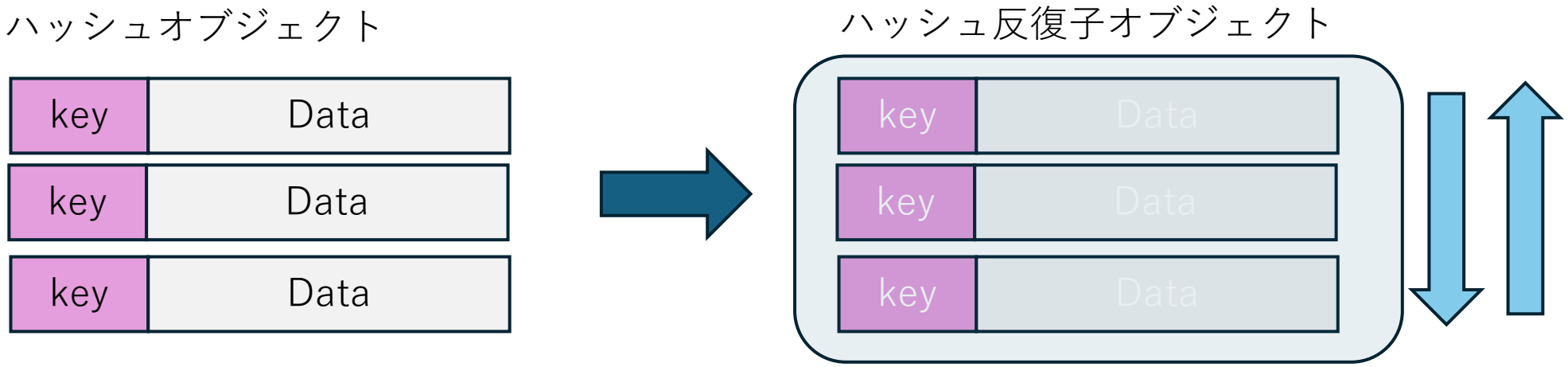
```
data wk1;
length SUBJID AENO $200. USED_DRUG $1000.;
 set AE;
 if 0 then set CM;
 if _N_ = 1 then do;
    declare hash
    h1(dataset:"CM",multidata:"Y");
    h1.definekey("SUBJID");
    h1.definedata("CMTRT","ACNAENO");
    h1.definedone();
  end;
call missing(of ACNAENO CMTRT);
 do while(h1.do_over() = 0);
    if ^missing(ACNAENO) then do;
      do i = 1 to count(ACNAENO,",") + 1;
        if AENO = scan(ACNAENO,i,",") then
        USED_DRUG=catx(",", USED_DRUG, CMTRT);
      end;
    end;
 end;
keep SUBJID AENO USED_DRUG;
run;
```

do_overメソッドの機能はfindメソッド＋find_nextメソッドをあわせたようなもので，これによって処理の冗長性が排除される

ハッシュ反復子オブジェクト

Hash Iterator Object

ハッシュ反復子オブジェクトにより、ハッシュオブジェクトのデータを前向きまたは逆向きのキー順序で取得できます。



ハッシュオブジェクトのメソッドはあくまで**key**に縛られた対象範囲しかもてないがハッシュ反復子オブジェクトは内部を自由にポインタ移動できる

```
data dm;
SUBJID="001";AGE=36;output;
SUBJID="002";AGE=40;output;
SUBJID="003";AGE=52;output;
run;
```

	SUBJID	AGE
1	001	36
2	002	40
3	003	52

```
data AGE_Criteria;
L_LIMIT=20;U_ULIMIT=29;cat="20 - 29";output;
L_LIMIT=30;U_ULIMIT=39;cat="30 - 39";output;
L_LIMIT=40;U_ULIMIT=49;cat="40 - 49";output;
L_LIMIT=50;U_ULIMIT=59;cat="50 - 59";output;
run;
```

	L_LIMIT	U_ULIMIT	cat
1	20	29	20 - 29
2	30	39	30 - 39
3	40	49	40 - 49
4	50	59	50 - 59

	SUBJID	AGE	AGECAT
1	001	36	30 - 39
2	002	40	40 - 49
3	003	52	50 - 59

データセットとしてCriteriaの情報があって
それをもって抽出や計算する場合，対象のデータセット
とCriteriraの中に直接的なリンクがなく
直積での処理となる

ハッシュオブジェクトを依り代にして，ハッシュ反復子オブジェクトは定義される

declare hiter 反復子オブジェクト名('基となるハッシュオブジェクト名')

```
data wk1;
  set dm;
  if 0 then set AGE_Criteria;
  if _N_ = 1 then do;
    declare hash h1(dataset:"AGE_Criteria",ordered:"Y");
    declare hiter hi1('h1');
    h1.definekey(all:"Y");
    h1.definedone();
 end;
   if hi1.first() = 0 then do;
    if L_LIMIT <= AGE <=  U_ULIMIT then AGECAT=cat;
   do while ( hi1.next()=0);
    if L_LIMIT <= AGE <=  U_ULIMIT then AGECAT=cat;
   end;
  end;
keep SUBJID AGE AGECAT;
run;
```

Firstメソッドで最初の位置にポインタ移動

L_LIMIT	U_ULIMIT	cat
20	29	20 - 29
30	39	30 - 39
40	49	40 - 49
50	59	50 - 59

Nextメソッドでハッシュ反復子オブジェクト内にレコードがある限り，順次走査し続ける

開始終了日の情報から持続期間をまとめる話

症例番号	カテゴリ	事象	持続期間	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
001	痛み	頭痛	2025/01/01～05																			
001	痛み	筋肉痛	2025/01/06～10																			
001	痛み	関節痛	2025/01/12～13																			
001	痛み	神経痛	2025/01/14～19																			
001	胃腸障害	胃炎	2025/01/01～08																			
001	胃腸障害	腸炎	2025/01/06～12																			
002	感染症	インフルエンザ	2025/01/02～08																			
002	感染症	感冒	2025/01/01～10																			



症例番号	カテゴリ	持続期間	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
001	痛み	2025/01/01～10																			
001	痛み	2025/01/12～19																			
001	胃腸障害	2025/01/01～12																			
002	感染症	2025/01/01～10																			

```
data wk1;
length ID cat term $30.;
format st en yymmdd10.;
ID="001";cat="痛み";term="頭痛";st="01JAN2025"d;en="05JAN2025"d;output;
ID="001";cat="痛み";term="筋肉痛";st="06JAN2025"d;en="10JAN2025"d;output;
ID="001";cat="痛み";term="関節痛";st="12JAN2025"d;en="13JAN2025"d;output;
ID="001";cat="痛み";term="神経痛";st="14JAN2025"d;en="19JAN2025"d;output;
ID="001";cat="胃腸障害";term="胃痛";st="01JAN2025"d;en="08JAN2025"d;output;
ID="001";cat="胃腸障害";term="腸炎";st="06JAN2025"d;en="12JAN2025"d;output;
ID="002";cat="感染症";term="インフルエンザ";st="02JAN2025"d;en="08JAN2025"d;output;
ID="002";cat="感染症";term="感冒";st="01JAN2025"d;en="10JAN2025"d;output;
run;
```

	ID	cat	term	st	en
1	001	痛み	頭痛	2025-01-01	2025-01-05
2	001	痛み	筋肉痛	2025-01-06	2025-01-10
3	001	痛み	関節痛	2025-01-12	2025-01-13
4	001	痛み	神経痛	2025-01-14	2025-01-19
5	001	胃腸障害	胃痛	2025-01-01	2025-01-08
6	001	胃腸障害	腸炎	2025-01-06	2025-01-12
7	002	感染症	インフルエンザ	2025-01-02	2025-01-08
8	002	感染症	感冒	2025-01-01	2025-01-10

```
proc sort data=wk1;
by ID cat st en;
run;

data wk2;
length ID cat $30. cat_st cat_en 8.;
format cat_st cat_en yymmdd10.;
set wk1;
if 0 then set wk1(rename=(st=_st en=_en));
if _N_ = 1 then do;
  declare hash h1(dataset:"wk1(rename=(st=_st en=_en))",multidata:"Y");
  h1.definekey("ID","cat");
  h1.definedata("_st","_en");
  h1.definedone();
end;
cat_st = st;
cat_en = en;
do while(h1.do_over() = 0);
if n(_st,_en) = 2 then do;
  if _st-1<=cat_en<=_en and cat_en <= _en then do;
                cat_en = _en;
                if _st < cat_st then cat_st = _st;
  end;
end;
end;
keep ID cat cat_st cat_en;
run;
```

	ID	cat	cat_st	cat_en
1	001	痛み	2025-01-01	2025-01-10
2	001	痛み	2025-01-06	2025-01-10
3	001	痛み	2025-01-12	2025-01-19
4	001	痛み	2025-01-14	2025-01-19
5	001	胃腸障害	2025-01-01	2025-01-12
6	001	胃腸障害	2025-01-06	2025-01-12
7	002	感染症	2025-01-01	2025-01-10
8	002	感染症	2025-01-01	2025-01-10

```
proc sort data=wk2 ;
by ID cat  cat_en cat_st;
run;

data wk3;
set wk2;
by ID cat  cat_en cat_st;
if first.cat_en;
run;
```

症例番号	カテゴリ	事象	持続期間	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
001	痛み	頭痛	2025/01/01～05																			
001	痛み	筋肉痛	2025/01/06～10																			
001	痛み	関節痛	2025/01/12～13																			
001	痛み	神経痛	2025/01/14～19																			
001	胃腸障害	胃炎	2025/01/01～08																			
001	胃腸障害	腸炎	2025/01/06～12																			
002	感染症	インフルエンザ	2025/01/02～08																			
002	感染症	感冒	2025/01/01～10																			

症例番号	カテゴリ	持続期間	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
001	痛み	2025/01/01～10																			
001	痛み	2025/01/12～19																			
001	胃腸障害	2025/01/01～12																			
002	感染症	2025/01/01～10																			

	ID	cat	cat_st	cat_en
1	001	痛み	2025-01-01	2025-01-10
2	001	痛み	2025-01-12	2025-01-19
3	001	胃腸障害	2025-01-01	2025-01-12
4	002	感染症	2025-01-01	2025-01-10

最近距離のマッチング処理の時などにハッシュ反復子オブジェクトはよく使う

```
data a;
set sashelp.class;
BMI = divide(WEIGHT* 703, HEIGHT**2 );
run;
```

ポンドとインチなのでBMIの計算式が普段と違う

	Name	Sex	Age	Height	Weight	BMI
1	Alfred	M	14	69	112.5	16.611531191
2	Alice	F	13	56.5	84	18.498551179
3	Barbara	F	13	65.3	98	16.156788436
4	Carol	F	14	62.8	102.5	18.270898414
5	Henry	M	14	63.5	102.5	17.870295741
6	James	M	12	57.3	83	17.771503583
7	Jane	F	12	59.8	84.5	16.611531191
8	Janet	F	15	62.5	112.5	20.2464
9	Jeffrey	M	13	62.5	84	15.117312
10	John	M	12	59	99.5	20.094369434
11	Joyce	F	11	51.3	50.5	13.490000722
12	Judy	F	14	64.3	90	15.3029757
13	Louise	F	12	56.3	77	17.077695295
14	Mary	F	15	66.5	112	17.804511278
15	Philip	M	16	72	150	20.341435185
16	Robert	M	12	64.8	128	21.429660113
17	Ronald	M	15	67	133	20.828469592
18	Thomas	M	11	57.5	85	18.073345936
19	William	M	15	66.5	112	17.804511278

同じクラスの中で自分と最もBMIの近い人を選ぶ処理を考える

```
%macro bmi();
  _BMI =  divide(_WEIGHT* 703, _HEIGHT**2 );
  diff=abs(round(BMI,10**-10) - round(_BMI,10**-10));
  if NAME ne _NAME and ( missing(mindiff) or diff <
mindiff) then do;
      Neighbour=_NAME;
      mindiff=diff;
      NBMI=_BMI;
  end;
%mend;
```

```
data wk2;
length NAME _NAME Neighbour $12. _HEIGHT _WEIGHT 8.;
set sashelp.class;
  if _N_ = 1 then do;
    declare hash h1(dataset:"sashelp.class(reNAME=(NAME=_NAME
HEIGHT=_HEIGHT WEIGHT=_WEIGHT))",ordered:"Y");
    declare hiter hi1('h1');
    h1.definekey(all:"Y");
    h1.definedone();
 end;
BMI = divide(WEIGHT* 703, HEIGHT**2 );
mindiff=.;
Neighbour="";
call missing(of _NAME _HEIGHT _WEIGHT _BMI);
if hi1.first() = 0 then do;
  %bmi();
  do while ( hi1.next()=0);
     %bmi();
   end;
end;
keep  NAME  Neighbour BMI NBMI mindiff;
run;
```

OBS	NAME	Neighbour	BMI	mindiff	NBMI
1	Alfred	Jane	16.6115	0.00000	16.6115
2	Alice	Carol	18.4986	0.22765	18.2709
3	Barbara	Alfred	16.1568	0.45474	16.6115
4	Carol	Thomas	18.2709	0.19755	18.0733
5	Henry	Mary	17.8703	0.06578	17.8045
6	James	Mary	17.7715	0.03301	17.8045
7	Jane	Alfred	16.6115	0.00000	16.6115
8	Janet	Philip	20.2464	0.09504	20.3414
9	Jeffrey	Judy	15.1173	0.18566	15.3030
10	John	Janet	20.0944	0.15203	20.2464
11	Joyce	Jeffrey	13.4900	1.62731	15.1173
12	Judy	Jeffrey	15.3030	0.18566	15.1173
13	Louise	Alfred	17.0777	0.46616	16.6115
14	Mary	William	17.8045	0.00000	17.8045
15	Philip	Janet	20.3414	0.09504	20.2464
16	Robert	Ronald	21.4297	0.60119	20.8285
17	Ronald	Philip	20.8285	0.48703	20.3414
18	Thomas	Carol	18.0733	0.19755	18.2709
19	William	Mary	17.8045	0.00000	17.8045

自分以外の人と総当たりでスコアを比較して
差の絶対値が小さければ入れ替えるということを繰り返す

もっともシンプルな総当たり探索.

ハッシュ反復子オブジェクトでのみ有効なメソッド

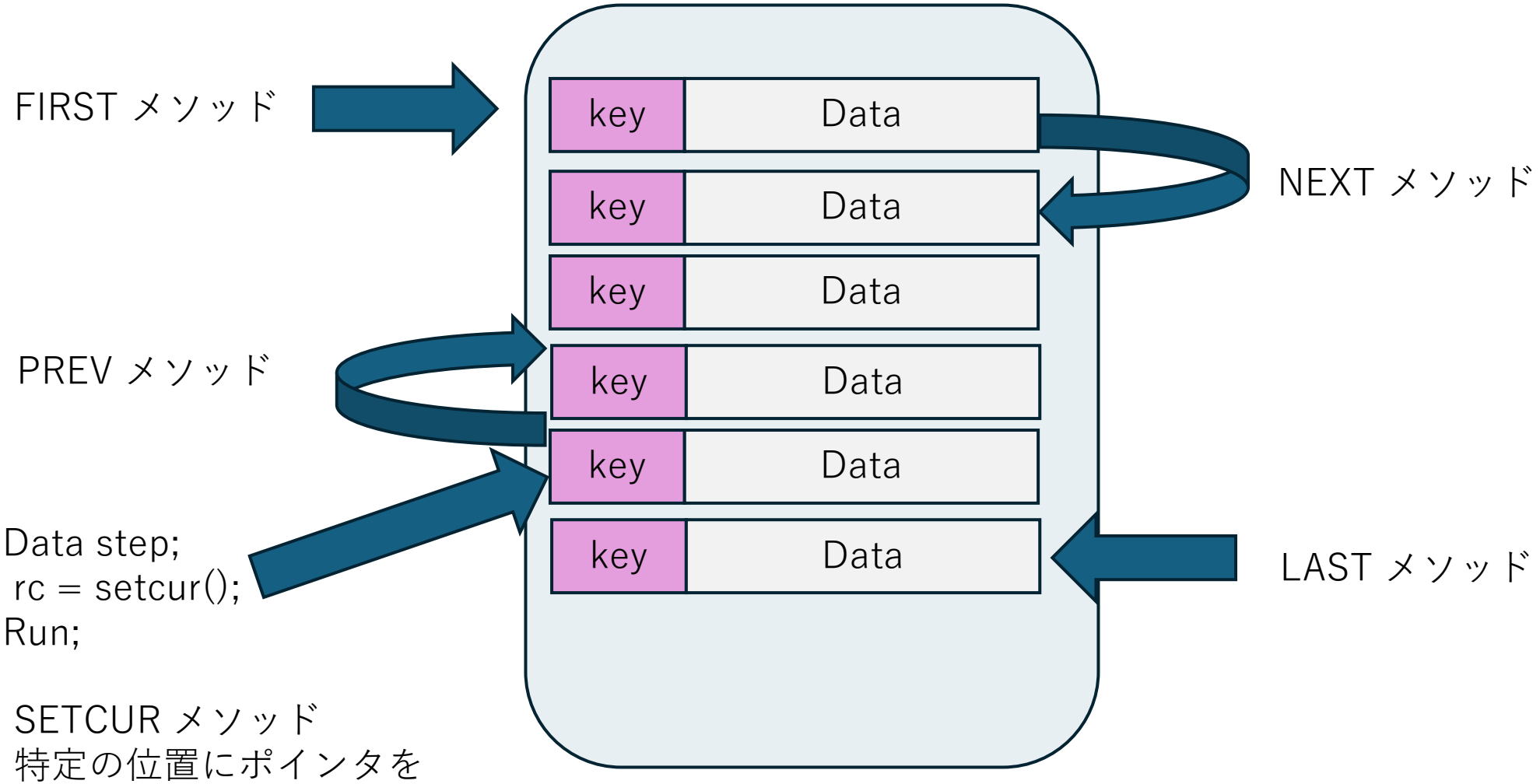
FIRSTメソッド
基となるハッシュオブジェクト内の最初のデータ項目を返して，そこにポインタ移動

LASTメソッド
基となるハッシュオブジェクト内の最後のデータ項目を返してそこにポインタ移動

NEXTメソッド
ハッシュ反復子オブジェクト（基のハッシュオブジェクト）において，現在位置の次の項目を返してそこにポインタ移動

PREVメソッド
ハッシュ反復子オブジェクト（基のハッシュオブジェクト）において，現在位置の前の項目を返してそこにポインタ移動

SETCURメソッド
基となるハッシュオブジェクトのkeyに基づいて，ポインタの位置を移動．値は返さない


FIRST メソッド
key
Data
key
Data
key
Data
key
Data
key
Data
key
Data
NEXT メソッド
PREV メソッド
Data step;
rc = setcur();
Run;
LAST メソッド
SETCUR メソッド
特定の位置にポインタを
動かす

結論

重複キーで設定したハッシュオブジェクトにおけるdo_overメソッドについて説明し
そこから，さらにキーの制約を受けずにハッシュオブジェクト内を自由にポインタ移動できる
ハッシュ反復子オブジェクトの基本を説明した

例示はいずれも単純なものであるが，工夫次第でいくらでも　複雑な処理の単純化，高速化が
可能なので，ぜひ勉強してみてください